

End-to-end gap map

PRIME.RDAGGER.ENDTOEND_GAPMAP.01 — round 462

Stefan Hamann

August 30, 2026

HIDDEN_GAP_FOUND

The claimed minimal path is neither source-faithful nor an end-to-end proof of the Riemann Hypothesis.

Abstract

This note audits every declaration and import behind `frequently_selected_augDualResolvent_ge_half  $\Rightarrow$  rh_of_frequently_selected`. The finite Loewner and filter algebra is sound and builds without `native_decide`, custom axioms, or partial definitions. The end-to-end claim nevertheless fails at three independent levels: (1) the Lean `ExactFold` window is not the Python production window measured in rounds 458–459; (2) the path assumes the unproved named bridge `SelectedACapPsdImpliesPlainReads` and consumes the `arch_elementwise_stabilization` sorry; and (3) its final conclusion is only nonnegativity of a custom `weilForm` on `GridElement`, with no spectral/zero side and no theorem connecting it to Mathlib’s zeta function or to the actual Riemann Hypothesis. No RH claim and no anti-RH claim is made.

1 Adversarial gap map

Glied	Datei:Zeile	Status	Risiko / genaue Aussage
Selected source	<code>Selected.lean:113--178</code>	HIDDEN GAP	<code>ExactPrimeSource</code> enumerates raw prime powers and <code>ExactFold</code> applies <code>foldedWindow</code> directly to those atoms. The production builder first assembles lag tents, takes a spectral density, and then folds/sign-splits a fixed grid.
Fold geometry	<code>Selected.lean:160--178;</code> <code>Source.lean:384--440</code>	HIDDEN GAP	Lean sets $\Delta = \log a/(m+1)$ and $L = 2(m+1)$, so $\Delta L = 2 \log a$ and the fold map is independent of m . Python uses $L = 2(m+1)-2$. Lean merges raw source nodes; Python has the spectral-grid support.
Moment carrier	<code>Selected.lean:240--270;</code> <code>FrequentlySelected.lean:97--111</code>	HIDDEN GAP	Lean <code>combHankel</code> is the moment Gram of separately folded, bare $\Lambda(p^r)$ weights. Python’s μ is the positive part after transforming the combined archimedean plus $2\Lambda(n)/\sqrt{n}$ lag vector. These are not the same matrix.
Border/budget	<code>Selected.lean:119--138,</code> <code>181--232;</code> <code>Canonical.lean:105--137</code>	OPEN	<code>ExactArch</code> , <code>ExactBorder</code> , and <code>ExactBudget</code> are values of the opaque <code>canonicalCompletion</code> ; <code>foldedWindow</code> passes u, B through unchanged. There is no executable Lean value to compare with <code>v958/r243</code> .
Selected cofinality	<code>Selected.lean:299--418</code>	PROVED	$a_k \rightarrow \infty$, $m_k \rightarrow \infty$, and $\Delta_k \rightarrow 0$ are proved. This does not repair the object mismatch: the window cone’s cap is computed from <code>ExactFold</code> ’s raw folded support, while m_k only appears in the separate read/extraction index.

Glied	Datei:Zeile	Status	Risiko / genaue Aussage
Mincut	FrequentlySelected.lean: 259--286	OPEN	frequently_selected_ augDualResolvent_ge_half is a <i>definition of an unproved Prop</i> , not a theorem or sorry. Rounds 458–459 provide only a finite Python census through $k = 12$, and on a different object.
$R^\dagger$ Loewner face	DualResolvent.lean: 360--387	PROVED	Rdagger_ge_half_iff_augmented_posSemidef is sorry-free finite matrix algebra, assuming dualZ.PosDef.
Real-window whitening	FrequentlySelected.lean: 118--251	PROVED	masterCap_posSemidef_iff_Rdagger_ge_half is a sorry-free congruence theorem under the explicit equations RepresentsLEnsembleReal. It identifies the cone with A_cap; it does not identify A_cap with fullRead.
Cone to A_{cap}	FrequentlySelected.lean: 370--381	PROVED	selectedWindowConeSemidef_implies_A_cap_posSemidef is proved from the whitening theorem.
A_{cap} to plain read	FrequentlySelected.lean: 382--409	HIDDEN GAP	SelectedACAPsdImpliesPlainReads is another unproved named Prop. It is an explicit hypothesis of the collapsed path. The claimed three-step minimal path omitted it.
Frequent extraction	FrequentlySelected.lean: 327--355, 410--448	PROVED modulo sorry	Filter arithmetic is proved: eventual coverage plus frequent positivity yields one good index. The proof calls elementwise_finite_stabilization.
Comb stabilization	Elementwise.lean: 360--492	PROVED	The mesh-free prime sum stabilizes for compactly supported GridElements. It concerns the custom read and does not prove the selected matrix/source identification above.
Arch stabilization	Elementwise.lean: 553--699	HIDDEN GAP	archRead and weilArchSide are independent opaque constants; their equality is the on-path sorry arch_elementwise_stabilization. It is not derivable in the current signature without replacing those constants by actual transcriptions.
Pole stabilization	Elementwise.lean: 522--708	PROVED	The native-mesh pole equality is definitional after the second-difference construction. Dyadic-independence is only a named Prop but is not consumed.
Custom Weil form	Elementwise.lean: 710--752	defined	weilForm=weilArchSide+weilCombSide+weilPoleSide. The arch term is opaque. The file explicitly says that the spectral/zero side is not part of this definition.
Mislabeled endpoint	FrequentlySelected.lean: 429--448	HIDDEN GAP	rh_of_frequently_selected concludes exactly forall f:GridElement, 0<=weilFormf. It has no conclusion about zeros of ζ . Its name is historical, not a formal RH theorem.
Dense-class extension	absent	HIDDEN GAP	No continuity/density theorem extends the autocorrelation GridElement class to the full admissible Weil test class.
Explicit formula	absent	HIDDEN GAP	No theorem identifies the custom weilForm with the standard Weil explicit formula, including its zero/spectral side.
Weil criterion $\Leftrightarrow$ RH	absent	HIDDEN GAP	No Mathlib or project theorem states the Weil criterion for Complex.zeta, identifies its nontrivial zeros, or concludes the genuine Riemann Hypothesis.

2 Numerical source-truth check

The companion probe enumerates the same complete set of prime powers $p^r \leq a^2$ on both sides, then evaluates the literal Lean comb fold and the production Python builder. Thus the discrepancy is not a table-cap artefact.

k	a	atoms	Lean S	Lean cap	Python S	Python cap
5	32	198	194	97	19	10
9	512	23150	23142	11571	71	36
10	1024	82267	82258	41129	79	40

At $k = 10$, the first three Lean literal `combHankel` entries (H_{00}, H_{01}, H_{11}) are

$$(1.04843042 \cdot 10^6, 869736.706, 811945.161).$$

The production μ -Gram entries are

$$(1.91210951, -1.15236093, 1.03066237).$$

The cap ratio is 1028.2. The corresponding ratios at $k = 5, 9$ are 9.7 and 321.4. Therefore the r459 `LEANFAM_EXACT_ALIVE(12)` census does not evaluate the Lean `selectedWindowConeSemidef` proposition.

3 Import graph and axiom audit

The actual import closure is:

- `FrequentlySelected` imports `Selected` and `GraphResolvent`.
- `Selected` imports `Canonical`, `DualResolvent`, and `Elementwise`.
- `Canonical` imports `Source`, `Closure`, `Augmented`, `PairBound`, and `Elementwise`.
- `GraphResolvent` imports `OneDefect` and `DualResolvent`; `OneDefect` imports `PivotCoordinate`, which imports `Haynsworth` and `Window`; these lead to `Inertia` and `Basic`.

`Source` imports `Window`; `Closure` and `Augmented` import `Inertia`; `PairBound` imports `Recursion`. The following modules are imported but their specialized theorems are not consumed by the final `FREQ` proof term: `GraphResolvent`, `OneDefect`, `PivotCoordinate`, `Haynsworth`, `Closure`, `Augmented`, and `PairBound`. `EdgeBalance.lean` and `Open.lean` were explicitly audited as requested but are not in this import closure and are off path.

There is no `native_decide`, `user axiom`, `admit`, or `partial` declaration on the path. The sorry-free matrix/filter theorems use only `propxt`, `Classical.choice`, and `Quot.sound`. The endpoint theorem additionally has `sorryAx` through `arch_elementwise_stabilization`. Noncomputability is ordinary classical choice; the semantic embeddings are the opaque constants `canonicalCompletion`, `archRead`, and `weilArchSide`, plus the named Props listed above.

4 Anatomy of all five sorry declarations

Declaration	Location	Path status	Closure assessment
<code>lstar_canonical</code>	<code>Canonical.lean</code> : 262--264	OFF PATH	Alternative rational-certificate L^* route. Not referenced by either <code>FREQ</code> endpoint theorem. Hard arithmetic problem; current certificate domain may be empty and must first be replaced by a faithful real domain.
<code>terminal_q_canonical</code>	<code>Canonical.lean</code> : 271--273	OFF PATH	Alternative terminal route. Not referenced by the <code>FREQ</code> proof. Hard cofinal arithmetic inequality; not closed by the finite census.

Declaration	Location	Path status	Closure assessment
<code>pair_terminal_dictionary</code>	<code>Canonical.lean:</code> 393--396	OFF PATH	Used only by <code>pair_closes_main</code> , not by the <code>FREQ</code> route. Potentially closable after a faithful v958 border/orthogonal-polynomial transcription; impossible from the current opaque completion alone.
<code>mainWindow_iff_builtFromPrimeSource</code>	<code>Source.lean:</code> 995--1008	OFF PATH	Historical opaque- <code>MainWindow</code> bridge, absent from the <code>FREQ</code> proof. Unprovable by design while both sides are opaque/free; closing it requires a semantic redefinition and all source transcriptions, not a local Lean lemma.
<code>arch_elementwise_stabilization</code>	<code>Elementwise.lean:</code> 675--697	HIDDEN GAP ON PATH	Consumed by <code>elementwise_finite_stabilization</code> and therefore by both <code>FREQ</code> endpoints. Classical analysis is plausible, but the current theorem relates two independent opaque constants. First define the actual tent integral and Weil arch pairing; then prove the Gauss/Mellin/digamma identity.

5 Verdict and architecture gate

The result is **HIDDEN GAP**, not `GAPMAP_CLEAN`. The reviewer must not accept the asserted three-arrow chain. Even granting the Mincut, three independent obligations remain before any RH theorem:

$$\begin{array}{l}
 \boxed{\text{SelectedACapPsdImpliesPlainReads}} \\
 + \quad \boxed{\text{arch_elementwise_stabilization}} \\
 + \quad \boxed{\text{custom Weil form} \iff \text{genuine RH}}.
 \end{array}$$

Before those, the selected Lean source itself must be reconciled with the production builder.

Part C is intentionally not proposed: the round contract permits a new full-solution architecture only if A/B are clean. The alias check against `rdagger_saturation.tex` is therefore not applicable; proposing another de Branges/Krein/Potapov architecture would conceal the more basic source and endpoint gaps found here.

Machine checks. `gapmap_probe.py--smoke`: 15/15 gates; source samples $k = 5, 9, 10$. `verify_gapmap.py`: declaration, census, source, and INVENTORY-stability gates; final line `GAPMAPVERIFIED`.

Claim boundary. Research documentation. No L^* claim, no $R^\dagger$ claim, no RH claim, and no anti-RH claim.